Segurança da comunicação entre *dongle* e servidor

1 Introdução

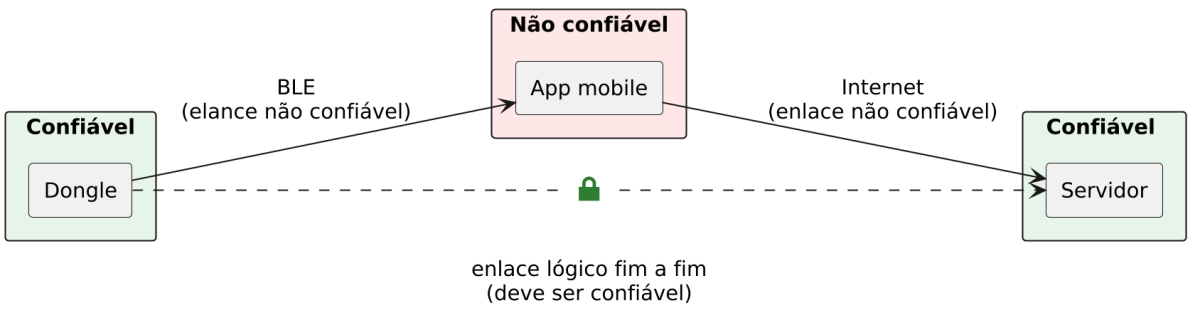
O propósito deste documento é modelar a implementação de uma comunicação com autenticidade entre *dongle* e um servidor externo, tendo como intermediário um *app mobile* que se comunica com o *dongle* por meio de *BLE* e não pode ser confiado. Os dados devem trafegar para o servidor de modo que, ao recebê-los, deve ser criptograficamente garantido que eles foram medidos a partir do dongle que se identifica como originador do dado, a fim de que não seja possível alterar o conteúdo das medições ou deliberadamente esconder informação entre duas medições diferentes sem que o servidor consiga detectar.

2 Problemas a resolver

O objetivo principal para o contexto deste trabalho é que as informações medidas pelo dongle que chegam no servidor possam ser confiadas e usadas para análises seguras a respeito do funcionamento do carro e como ele está sendo conduzido. Assim, deve-se garantir autenticidade, ou seja, todo registro recebido pelo servidor deve ter sido gerado pelo dongle associado a ele; integridade, ou seja, o dado que é recebido pelo servidor não pode ter sido alterado; unicidade, ou seja, os registros devem ser apenas uma única vez e a ordem em que eles foram gerados deve ser mantida; e completude detectável, ou seja, o servidor deve saber exatamente se e quais registros estão faltando ao sincronizar com o *dongle*.

Nesse contexto, só é poossível confiar plenamente no dongle e no servidor. Assim, o app e os enlaces BLE e internet são modelados como não confiáveis, conforme a Figura 1.

Figura 1 – Fronteiras de confiança entre *dongle*, *app* e servidor



Fonte: elaborado pelo autor.

2.1 Modelagem de ataques

Existem diversos adversários e ataques que podem ser modelados no cenário, envolvendo os diferentes componentes e ligações entre eles. É possível dividir os ataques em três principais categorias.

Dentro das categorias de ataque, também é possível definir cinco tipos principais de adversários que aparecem entre elas:

- (AD1) o terceiro perto de carro que pode ter acesso ao enlace de rádio do BLE, cujo objetivo é espionar ou injetar dados;
- (AD2) o próprio motorista que tem acesso ao app original do sistema, cujo objetivo é esconder, alterar medidas do dongle ou enviar as medidas do dongle para terceiros não autorizados;
- (AD3) um app de terceiro no celular pareado que pode acessar as características BLE, cujo objetivo é coletar as informações medidas para outros propósitos;
- (AD4) um atacante na rede entre o celular e o servidor, cujo objetivo é ler ou alterar dados no caminho para o servidor;
- (AD5) um invasor no servidor que tem acesso ao banco de dados e tudo que está armazenado, cujo objetivo pode ser apagar ou forjar dados.

Para especificar os ataques, será utilizada uma metodologia de modelagem que define o adversário, a pré-condição do ataque, o método a partir do qual o ataque é executado, a vulnerabilidade que ele explora, o alvo que está sendo atacado, e, por fim, o efeito que o ataque causa.

2.1.1 Comunicação wireless

Esses são ataques que se concentram tanto no enlace BLE entre o app e dongle quanto na conexão de rede entre o app e o server. Dentre eles, estão sniffing, replay, Man In The Middle (MITM), acesso direto ao dongle por um terceiro e negação de serviço do enlace.

Abaixo estão modelados os ataques do ponto de vista do BLE, mas os mesmos princípios podem ser pensados para um adverário do tipo AD4 que interfere na comunicação entre o app e o servidor.

Um exemplo de ataque dessa categoria é o sniffing (A1.1), no qual o adversário do tipo AD1 pode querer obter os dados do motorista, afetando sua privacidade. A pré-condição é ter um sniffer ao alcance do sinal do BLE, cujo enlace não é cifrado. O método a partir do qual ele é feito é seguir a conexão e recuperar tudo que passa pelas características do dongle de maneira passiva. A vulnerabilidade explorada é a falta de cifração nas características

BLE, o alvo são os dados transferidos pelo enlace, e o efeito é o vazamento de dados medidos pelo dongle.

No ataque de *replay* (A1.2), o adversário do tipo AD1 quer repetir comandos ou respostas para alterar o estado do dongle ou enganar o app. A pré-condição é ter capturado o tráfego por *sniffing* e conseguir se conectar ao dongle ou se passar por ele. O método consiste em reescrever na característica de recepção quadros gravados anteriormente, como `AT#DIAG 1`, `AT#PID` ou `ATZ`, ou em se anunciar como o dongle e devolver respostas antigas ao app. A vulnerabilidade explorada é a ausência de autenticação da conexão e de garantia de frescor das mensagens, somada ao fato de o app não verificar a identidade do dongle. O alvo são o modo e a configuração do dongle e os dados exibidos pelo app, e o efeito é uma coleta pausada ou reconfigurada sem o conhecimento do motorista, ou um painel com valores falsos.

Já no *Man In The Middle* (A1.3), o adversário do tipo AD1 quer ler e alterar em tempo real o que app e dongle trocam. A pré-condição é estar presente quando o app se conecta ou durante o pareamento, com dois rádios BLE. O método é conectar-se ao dongle, anunciar um clone dele ao app e repassar todo o tráfego entre os dois, alterando valores ou inserindo comandos no caminho. A vulnerabilidade explorada é que nenhum dos lados autentica o outro no enlace, o alvo é a integridade de comandos e respostas, incluindo as requisições ao barramento CAN no modo diagnóstico, e o efeito é um painel adulterado e requisições injetadas no carro.

Por fim, no acesso direto (A1.4), o adversário do tipo AD1 quer usar o dongle como se fosse o app. A pré-condição é que o dongle aceite conexões de qualquer dispositivo, e o método é simplesmente conectar-se e escrever comandos na característica de recepção, inclusive requisições ao barramento CAN no modo diagnóstico. A vulnerabilidade explorada é a falta de autenticação da conexão, agravada pelo dongle aceitar uma segunda conexão simultânea. O alvo são os dados medidos, o modo diagnóstico e a configuração do dongle, e o efeito é a leitura dos dados, a pausa da coleta e consultas ao veículo feitas em nome do dono.

Na negação de serviço do enlace (A1.5), o adversário do tipo AD1 quer impedir que o app se comunique com o dongle, atacando a disponibilidade da conexão. A pré-condição é estar ao alcance do sinal BLE do dongle. O método é conectar-se ao dongle assim que ele começa a se anunciar e manter a conexão aberta, ocupando as conexões que ele aceita, de modo que o app oficial não consiga se conectar. A vulnerabilidade explorada é que o dongle aceita conexões de qualquer dispositivo e tem um número limitado delas. O alvo é a disponibilidade do enlace BLE e, por consequência, a sincronização dos registros com o servidor, e o efeito é um dongle inacessível ao app: o painel deixa de ser atualizado e o servidor recebe um intervalo sem dados que não é causado pelo motorista.

2.1.2 Comunicação entre carro e dongle

Já esses ataques acontecem no barramento CAN entre o carro e o dongle, ou seja, o atacante pode usar algum elemento externo para simular a CAN do veículo por inteiro ou para sobrescrever um determinado PID a fim de manipular leituras.

No ataque de ECU emulada (A2.1), o adversário do tipo AD2 quer que a seguradora receba uma condução melhor do que a real. A pré-condição é ter acesso físico à porta OBD e um emulador ou *gateway* CAN, sendo que o próprio ECUSim deste projeto serve como prova de conceito. O método é ligar o dongle no emulador, ou colocar um *gateway* entre a porta e o dongle que repassa as requisições ao carro e altera apenas as respostas escolhidas, como limitar a velocidade reportada. A vulnerabilidade explorada é que as respostas OBD-II não são autenticadas, de modo que o dongle confia em quem responder. O alvo é a veracidade dos dados medidos antes de serem assinados, e o efeito são registros autênticos com conteúdo falso: a assinatura prova que o dongle mediu, mas não que o carro disse aquilo.

Na resposta concorrente no barramento (A2.2), o adversário do tipo AD2 quer sobrescrever apenas alguns PIDs sem retirar o carro do barramento. A pré-condição é ter um dispositivo ligado ao CAN capaz de responder mais rápido que a ECU. O método é escutar as requisições funcionais, enviadas para o identificador 0x7DF, e responder em 0x7E8 antes da ECU real. A vulnerabilidade explorada é a mesma do ataque anterior, somada ao fato de o dongle aceitar a primeira resposta da faixa 0x7E8–0x7EF sem endereçar ECUs específicas. O alvo é a veracidade dos PIDs escolhidos, e o efeito, além dos registros autênticos com conteúdo falso, é um painel e um diagnóstico enganados.

2.1.3 App malicioso

Como o aplicativo será instalado no aparelho do usuário do dongle, um aparelho com *root* tem total controle do funcionamento do app e pode manipulá-lo para gerar ataques ao sistema. Esses ataques podem manipular o processo de envio dos dados entre o app e o servidor a fim de fazer com que as informações do servidor não correspondam com o que foi realmente medido pelo dongle no carro. O principal adversário nesse contexto é o do tipo AD2, que controla o app e pode querer forjar cenários de medição que vão de encontra aos interesses da entidade no servidor externo, como uma seguradora ou gerenciadora de frotas.

Na exfiltração (A3.1), o adversário do tipo AD3, ou um SDK de terceiros embutido no próprio app, quer vender ou usar para outros fins os dados de condução e de posição do motorista. A pré-condição é ter permissão de Bluetooth e de internet no celular pareado com o dongle. O método é conectar-se ao dongle quando o app oficial não está conectado e ler os dados medidos, ou ler os registros e posições que passam pelo app, enviando tudo a outro servidor. A vulnerabilidade explorada é que os dados não são cifrados na camada de

aplicação e que a autorização é feita por aparelho, e não por app. O alvo é a privacidade do motorista, e o efeito é o vazamento de dados pessoais e o rastreamento do veículo.

Já na omissão de mensagens do servidor (A3.2), o adversário do tipo AD2, por meio de um app modificado, quer degradar a coleta sem que a culpa recaia sobre o app. A pré-condição é que o app seja o único caminho entre o servidor e o dongle. O método é receber do servidor o desafio, a confirmação de recebimento ou o *token* de diagnóstico e não repassá-los ao dongle, ou repassar apenas alguns. A vulnerabilidade explorada é que as mensagens do servidor não têm confirmação de entrega ao dongle. O alvo é a recuperabilidade dos registros, a âncora de tempo e a troca de época, e o efeito é que, sem confirmações, a memória do dongle enche e ele descarta registros antigos; sem desafio, não há âncora de tempo nem época nova; e, sem *token*, o diagnóstico é negado.

Na alteração de medições (A3.3), o adversário do tipo AD2 quer melhorar o próprio histórico. A pré-condição é controlar o app, ou um *proxy* com uma autoridade certificadora instalada no celular, e o método é modificar bytes de um bloco antes de enviá-lo ao servidor. A vulnerabilidade explorada é a ausência de integridade fim a fim, o alvo é a integridade dos registros, e o efeito é a aceitação de dados falsos.

Na repetição de medições (A3.4), o adversário do tipo AD2 quer repetir uma viagem tranquila. A pré-condição é ter gravado blocos antigos, e o método é reenviar ao servidor blocos já aceitos. A vulnerabilidade explorada é a falta de garantia de unicidade, o alvo é a unicidade dos registros, e o efeito são dados duplicados no servidor.

Na reordenação das medidas enviadas (A3.5), o adversário do tipo AD2 quer deslocar eventos no tempo. A pré-condição é controlar o app, e o método é enviar os blocos fora de ordem. A vulnerabilidade explorada é que a ordem dos registros não é garantida, o alvo é a ordem dos registros, e o efeito é uma linha do tempo falsa.

No descarte de medições intermediários (A3.6), o adversário do tipo AD2 quer apagar um evento específico, como uma frenagem brusca. A pré-condição é controlar o app, e o método é deixar de enviar os blocos escolhidos. A vulnerabilidade explorada é a falta de completude, o alvo é a completude dos registros, e o efeito é uma lacuna silenciosa nos dados do servidor.

Ao omitir as últimas medições (A3.7), o adversário do tipo AD2 quer omitir a última viagem. A pré-condição é controlar o app, e o método é parar de enviar blocos antes do último. A vulnerabilidade explorada é que o servidor não conhece o número de sequência do último registro gerado, o alvo é a completude dos registros, e o efeito é uma perda de dados invisível ao servidor.

2.2 Definição de escopo e mitigação de ataques

Tendo em vista todos esses possíveis ataques, é necessário alinhar o escopo do projeto de acordo com o objetivo que ele se propões a cumprir. Como a ideia é fazer com que os dados cheguem no servidor de maneira confiável e completa, foram priorizados os ataques

da categoria 2.1.3, sendo o objetivo deste documento explicar em detalhe um protocolo que garante, nesse cenário de desconfiança no canal e no aplicativo que intermedia o envio dos dados, como garantir autenticidade, integridade, unicidade e completude detectável dos dados medidos pelo dongle.

Além disso, para complementar a segurança da comunicação fim a fim, também será abordado o uso de segunrança entre os enlaces BLE e internet para abordar os ataques modelados na seção 2.1.1. Como já existem protocolos estabelecidos para garantir essa segurança, a ideia é usá-los para garantir confidenciabilidade na comunicação: para o enlace entre o dongle e o app, será utilizada a técnica LESC (LE Secure Connections) do BLE; já para o enlace entre app e servidor será utilizado o protocolo HTTPS/TLS, que é amplamente utilizado para garantir a segurança de comunicação entre clientes e servidores na internet.

Fica como limitação do projeto a mitigação de ataques que envolvam o barramento CAN do veículo, como os da categoria 2.1.2, pois o escopo do projeto envolve apenas os dados medidos pelo dongle, sem abordar a garantir de que o dongle está realmente conectado em uma rede CAN válida, o que deve ser garantido externamente. Alguns exemplos de mitigação para isso é incoporar o dongle dentro da arquitetura do carro e não exposto na conexão OBD2 interna, configurar alertas de desconexão para entender se o dongle foi retirado a fim de ser colocado em um simulador, ou cruzar a medida com sensores externos de velocidade e localização instalados externamente à rede CAN.

3 Segurança na comunicação sem fio

3.1 BLE LESC

O LE Secure Connections (LESC) é um método de pareamento introduzido no Bluetooth 4.2 para substituir o esquema de chave temporária de baixa entropia no pareamento legado do BLE, que era facilmente quebrada por força bruca. Para isso, o LESC estabelece a chave do enlace com uma troca de chaves Diffie-Hellman em curva elíptica (ECDH) na curva P-256. Com essa chave de enlace, cada um dos aparelhos derivam uma chave de sessão com a qual o enlace é cifrado e autenticado usando AES-CCM, o que garante confidenciabilidade e integridade na comunicação. Quando os dois aparelhos são pareados, ou seja, geram a chave de enlace válida,é feito o *bonding*: a chave de sessão é salva em cada um dos aparelhos para futuras conexões, de modo que não seja necessário repetir o processo de pareamento.

Esse pareamento é feito com um total de 3 fases. Na primeira fase, os aparelhos declaram suas capacidades e requisitos de autenticação como, por exemplo, se têm capacidade de entrada e saída e se suportam LESC. Trocadas as capacidades, os dispositivos decidem se continuam o pareamento e é escolhido um método de associação, que estão especificados na

tabela 3.1, na qual considera-se implícita a pré-condição de que ambos os lados suportam LESC.

Tabela 1 – Métodos de associação do LESC

Método	**Pré-condição**	**Descrição**
Just Works	-	Não há autenticação do outro lado, apenas a troca de chaves ECDH. Vulnerável a MITM.
Numeric Comparison	Ambos têm display e entrada	Um número é gerado e exibido em ambos os lados, que devem confirmar se são iguais.
Passkey Entry	Um tem display e outro tem entrada	Um lado gera um número de 6 dígitos e o outro deve digitá-lo.
OOB (Out of Band)	Canal seguro externo entre os dispositivos	Dados de autenticação são trocados por um canal seguro externo, como NFC.

3.2 HTTPS/TLS

O mesmo problema de confidenciabilidade e integridade de comunicação sem fio que é identificado entre o dongle e o app, também pode ser identificado quando o app vai se comunicar com o servidor: deve haver confidenciabilidade, autenticidade e integridade na troca de pacotes pela rede. Para isso, o servidor deve ter um certificado TLS válido a fim de que seja possível estabelecer uma troca de chaves com autenticidade. Em seguida, com a chave estabelecida, há garantia de integridade, confidencialidade e autenticidade entre a conexão app e servidor.

4 Segurança na comunicação fim a fim

Esta seção tem como propósito modelar como deve ser feita a comunicação fim a fim, entre dongle e servidor, garantindo que haja todas as propriedades de segurança previstas no projeto. A ideia é comparar com benchmarks duas implementações diferentes: usando MAC (Message Authentication Code) com chave simétrica pré-compartilhada (PSK) e usando assinatura digital via chave assimétrica. Em ambas as formas, a ideia é o dongle conseguir provar criptograficamente que validou os blocos antes de enviá-los para o servidor e, quando esses blocos chegarem no seu destino, o servidor consiga validar que os blocos que chegaram pela rede são de fato gerados pelo dongle.

Para entender a necessidade dessa modelagem, é importante deixar claro o motivo das camadas de LESC e HTTPS na comunicação não serem suficientes. Esses dois mecanismos garantem segurança na comunição entre dongle e app e entre app e servidor, respectivamente, mas não conseguem garantir que pacotes realmente foram gerados pelo dongle, uma vez que eles serão transmitidos por meio de um intermediário que tem controle total sobre o conteúdo.

4.1 Geração de chaves e identificação do dongle

Na fabricação do dongle, serão gerados um *DevId* de 8 bytes e duas chaves K_{auth} e K_{rec} de 32 bytes cada. Cada um desses elementos é sorteado usando forte entropia ao fabricar o dongle. As chaves são gravadas na NVS do dongle e registradas em um serviço de KMS (Key Management System) em um servidor separado (S_{KMS}), trafegando na rede criptografadas com uma chave pública fornecida por S_{KMS}; dessa maneira, a chave fica em claro apenas por pouco tempo na memória da estação de fabricação.

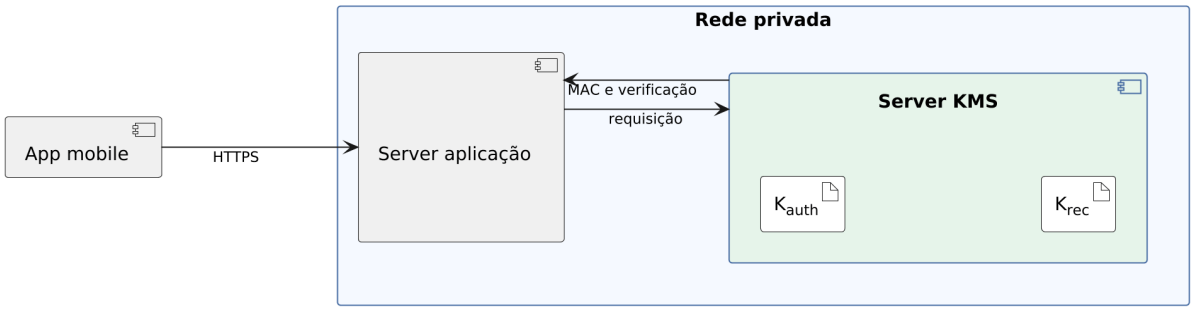
O uso de um KMS é importante pois permite que o servidor consiga verificar e gerar MACs sem que a chave precise estar na memória do servidor de aplicação (S_{App}), que é um alvo de mais fácil acesso e, portanto, com maior risco de comprometimento, diminuindo a superfície de ataque para adversários do tipo AD5. Ainda há superfície de ataque no servidor do KMS, mas ele não é exposto a internet e pode ter um controle de acesso bem mais rídigo.

A escolha de usar duas chaves é para que K_{auth} seja usada para criar MAC nas mensagens de autenticação entre o servidor e o dongle, enquanto K_{rec} fica apenas para gerar MAC dos blocos de dados que saem do dongle até o servidor. Assim, é possível gerenciar as permissões do KMS para que o servidor de aplicação consiga gerar e verificar MACs com K_{auth}, mas consiga apenas verificar MACs com K_{rec}. Assim, mesmo que S_{App} seja comprometido, um atacante não consegue gerar MACs válidos para um dongle sem também comprometer o S_{KMS}.

Enquanto do lado do servidor o KMS garante a proteção das chaves, o dongle conta com o armazenamento das chaves no NVS (Non-Volatile Storage) com excriptação. Para garantir maior segurança desse encriptação, também conta com: Secure Boot V2 para rodar apenas *firmware* assinado e não permitir injeção arbitrária de código; *flash encryption* para cifrar as partições da flash e a chave de encriptação do NVS; *eFuse*, memória gravável e não editável que guarda configurações da placa como o bloquear do uso de JTAG e download pela UART, evitando depuração, e também guardar a chave de encriptação da flash, contando com bloqueio total de leitura. Mesmo com todas essas camadas de proteção, ainda há risco de ataques com atacantes extremamente qualificados.

Uma evolução para guardar melhor as chaves é o uso de um elemento seguro de hardware, como o chip ATECC608B, para armazenar as chaves fora da ESP. Como, mesmo ainda muito mais difícil, ainda é possível extrair chaves de elementos seguros, não vale a pena o seu uso dentro desse projeto, pois agrega apenas em aumento de custo para um ataque de recuperação de chave e não configura uma melhoria academicamente relevante em relação a segurança dentro do escopo deste trabalho.

Figura 2 – Interação entre o app, o servidor de aplicação e o servidor KMS



Fonte: elaborado pelo autor.

4.2 Garantia das propriedades de segurança

A coleta no dongle ocorre de maneira offline-first, como mencionado em Investigação-3.pdf. Para que não seja possível alterar blocos guardados no sistema de arquivos do dongle, assim que o buffer em memória enche o bloco vai ser guardados na memória *flash*, é calculada uma tag de autenticação de 32 bytes via HMAC-SHA256 a partir da chave K_{rec}, que é salva concatenada ao bloco de dados. Desse modo, caso o dado seja alterado enquanto está na memória, a tag se tornará inválida.

Quando um bloco chega no servidor é possível verificar a tag a partir de uma chamada ao KMS para validar a mensagem e, em seguida, salvar o bloco decodificado. Desse modo, é possível garantir integridade e autenticidade do bloco no servidor. Esse desenho garante que ataques do tipo A3.3 da seção 2.1.3 não consigam alterar blocos antigos antes de enviar ao servidor e também garante que não seja possível fabricar novos blocos.

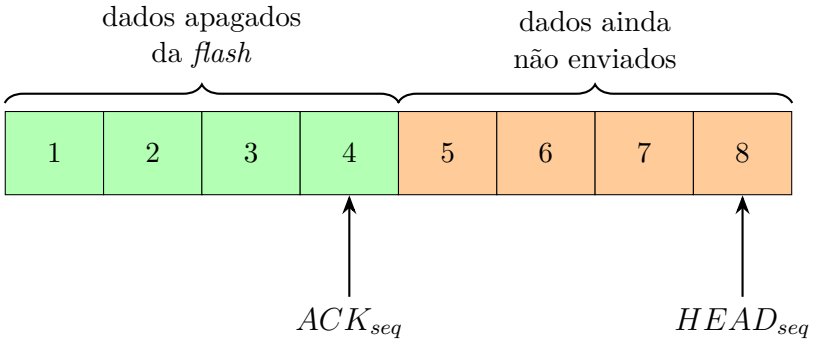
Para que seja possível garantir unicidade, quando um bloco cujo MAC seja válido é recebido pelo servidor, é extraída a *seq* do cabeçado desse bloco. O servidor verifica se um bloco com o mesmo valor de *seq* foi recebido, descartando o bloco em caso positivo. Assim, garante-se que um atacante não consiga repetir blocos antigos para alterar os registros no servidor, como foi modelado para o adversário do tivo AD2 no ataque A3.4 da seção 2.1.3.

Para que seja possível garantir completude detectável, é necessário que o servidor consiga identificar com exatidão qual foi o número *seq* no último bloco recebido, ACK_{seq}, a fim de que seja possível comparar com o *seq* do último bloco medido pelo dongle, o $HEAD_{seq}$, sendo a diferença $HEAD_{seq} - ACK_{seq}$ entre esses valores a quantidade de blocos que estão com envio pendente para o server; ao mesmo tempo, todos os blocos com $seq \leq ACK_{seq}$ já foram recebidos e processados, podendo ser excluídos da *flash*. Para garantir sequência e ordem, o servidor apenas aceita pacotes cujo valor de seq é exatamente $ACK_{seq} + 1$, descartando pacotes fora de ordem (Go-Back-N). Essa dinâmica é ilustrada na Figura 3.

Assim, com essa modelagem, é possível garantir ordem e completude detectável, impedindo que as medidas sejam reordenadas ou que sejam criadas lacunas entre diferentes

medidas.

Figura 3 – Fila de blocos no dongle e a posição de ACK_{seq} e $HEAD_{seq}$



Fonte: elaborado pelo autor.

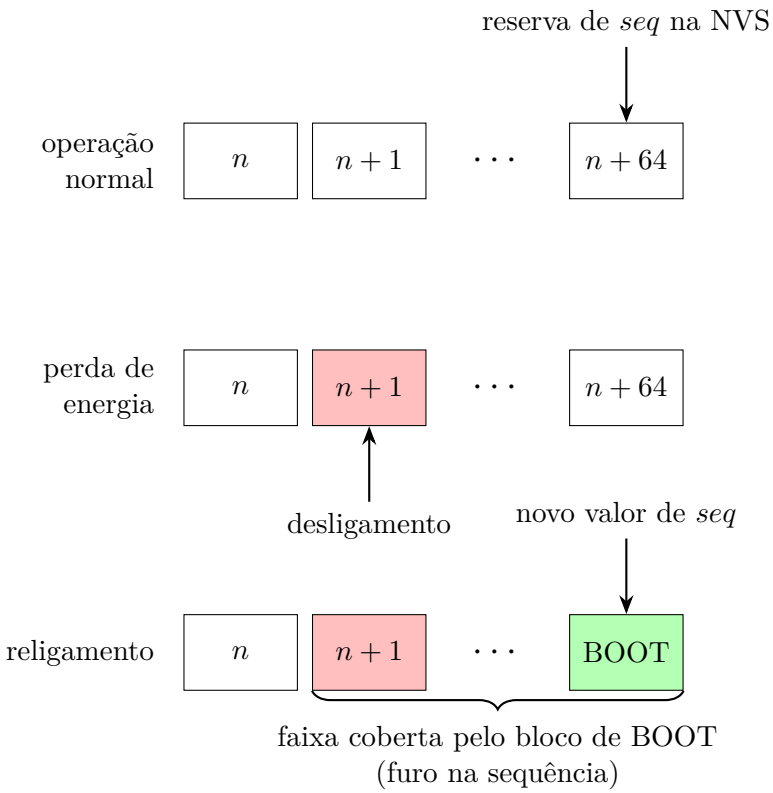
Uma limitação real desse modelo vem da dependência do app para conexão com a internet. Um app malicioso pode deliberadamente parar de encaminhar comunicações entre o dongle e o server, ocasionando na omissão de medidas e inviabilização da conexão via handshake, como está modelado nos ataques A3.2 e A3.7 da seção 2.1.3. Mesmo que não seja possível mitigar diretamente esse tipo de ataque, a falta dos dados é detectável do lado do servidor, podendo gerar um alerta.

4.3 Riscos de sincronização

O fato de o servidor apenas aceitar valores de *seq* exatamente iguais a $ACK_{seq} + 1$ pode gerar problema de sincronização em dois casos diferentes: quando ocorre perda de energia, o valor de *seq* é perdido, pois ele é armazenado apenas em RAM, uma vez que seu valor muda muito e desgastaria bastante a NVS reescreve-lo a todo bloco gerado; quando um bloco salvo na *flash* é corrompido, a verificação de HMAC falha no servidor e ele é rejeitado e, como não há bloco válido com aquele número de *seq*, o servidor não consegue mais receber blocos. Para abordar esses problemas, são criados dois blocos especiais de BOOT e de LOSS.

Para o bloco de BOOT, é necessário manter uma reserva de *seq* na NVS para que o contador retome a partir do valor da reserva sempre que perder alimentação. Uma sugestão para esse valor é sempre armazenar o valor de $seq + 64$ na NVS a cada 64 blocos gerados e gravados na *flash*. Sempre que o dongle perder energia, ao ser religado ele deve ler o valor da reserva *seq* da NVS, retomar a contagem a partir dele e, por fim, gerar um bloco de BOOT declarando que há furo na sequência entre o último bloco salvo na *flash* e o valor que está na reserva de *seq*.

Figura 4 – Reserva de *seq* na NVS e geração do bloco de BOOT após perda de energia



Fonte: elaborado pelo autor.

Para o caso de corrompimento, é necessário verificar se houve corrompimento no bloco sempre antes de enviá-lo ao app para ir ao servidor e, caso seja detectado, deve ser gerado um bloco de LOSS que ocupa o próprio *seq* do bloco corrompido (ou, se mais de um bloco for afetado, declara explicitamente a faixa de *seq* perdida), de modo que o servidor receba um bloco válido naquela posição, registre-o como LOSS e a sequência volte a avançar. Seria possível verificar a integridade do bloco apenas calculando o hash, mas isso é computacionalmente custoso para recalcular em todo bloco. Por isso, é mais eficiente salvar um CRC32, calculado sobre os dados e a tag HMAC, em todo bloco e verificar esse CRC32 antes de mandar para o app. O uso do CRC32 adiciona apenas 4 bytes por bloco, que é justificável depois das otimizações feitas anteriormente.

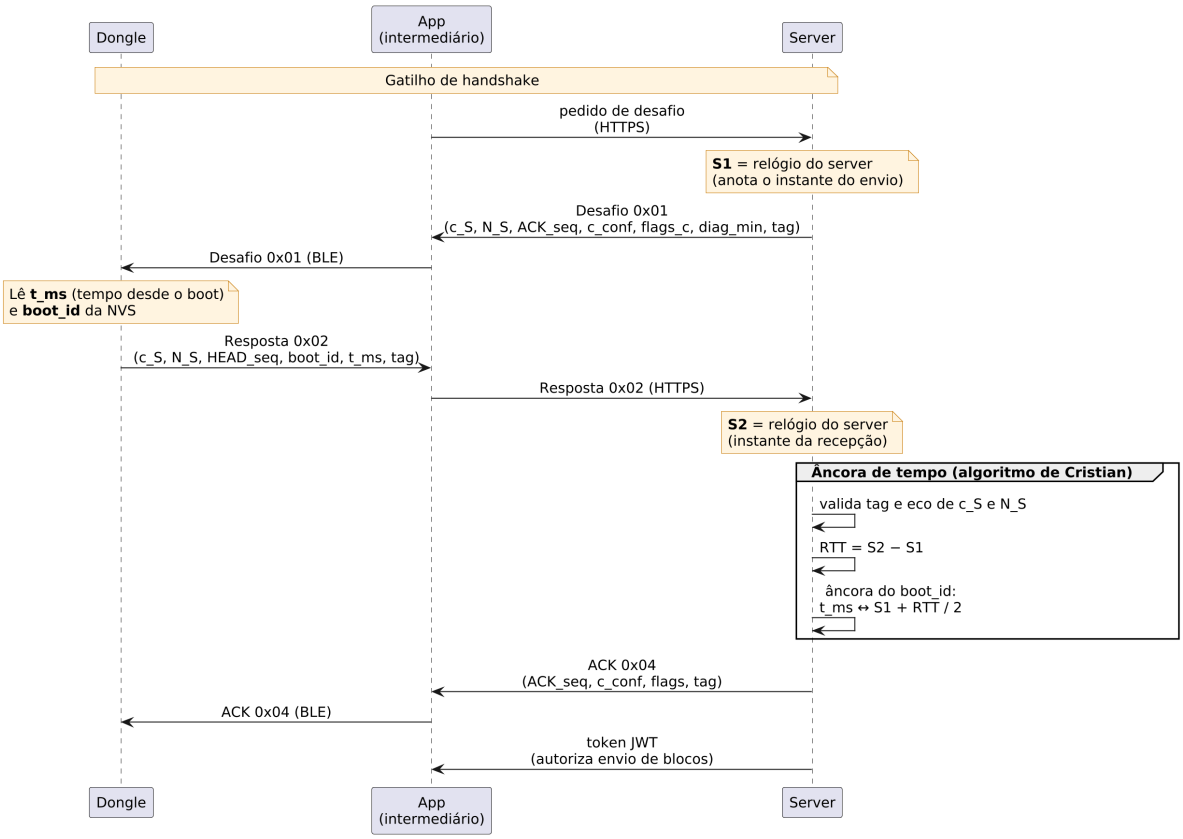
4.4 Especificação do processo de handshake

Quando o dongle começa sua operação principal, ele salva um $boot_id$ na NVS que serve até ele deixar de ser alimentado ou até entrar e sair de deep sleep. Enquanto o dongle estiver offline, a coleta dos PIDs de telemetria vai continuar ocorrendo como esperado, com blocos passando buffer em RAM e indo para a memória *flash* com suas tags de autenticação e um CRC32.

Assim que há um pedido de handshake do dongle ou assim que o app se conecta com a internet, o *handshake*, utilizando os parâmetros definidos na tabela 2, começa e segue a seguinte sequência de passos, abstraindo a intermediação do app:

1. Dongle pede que S_{App} envie um desafio
2. Server anota $S1$
3. Server envia um bloco desafio tipo 0x01 para o dongle contendo: c_S, N_S, ACK_{seq}, c_{conf}, $flags_c$, $diag_min$ e tag (47 bytes) com destino ao dongle
4. Dongle responde um bloco tipo 0x02 contendo: $HEAD_{seq}$, $boot_id$, t_{ms}, e tag (46 bytes) com destino ao server
5. Com $S2$, o server fecha a âncora de tempo para $boot_id$ segundo algoritmo de Christian
6. Server envia bloco ACK tipo 0x04 contendo: ACK_{seq}, c_{conf}, $flags_a$, e tag (25 bytes) com destino ao dongle
7. Server envia, com destino ao app, um token JWT válido para receber blocos do app.

Figura 5 – Diagrama de sequência do *handshake* e da âncora de tempo



Fonte: elaborado pelo autor.

Tabela 2 – Parâmetros do processo de *handshake*

Parâmetro	**Descrição**	**Origem**	**Bytes**
c_S	Contador de desafios associado ao dongle pelo seu *DevId*	S_{App}	4
N_S	*Nonce* que identifica o desafio	S_{App}	16
ACK_{seq}	Fronteira que indica o último *seq* recebido pelo server	S_{App}	4
c_{conf}	c_S do último desafio aceito para conferência	S_{App}	4
$flags_c$	Bit 0: indica se ativa modo dignóstico; Outros bits reservados	S_{App}	1
$diag_min$	Tempo em minutos que o dispositivo ficará em modo diagnóstico	S_{App}	2
$HEAD_seq$	Valor de *seq* do último bloco gravado no LittleFS	Dongle	4
boot_id	O valor de ID do *boot* atual do dongle	Dongle	2
t_{ms}	Tempo em milissegundos desde o boot no instante da resposta	Dongle	4
$flags_a$	Bit 0: indica falha na sincronização de tempo, *handshake* deve ser repetido; Outros bits reservados	Dongle	1
tag	Tag calculada via HMAC-SHA256	Dongle ou S_{App}	16

Para emitir o desafio, o server incrementa c_S, sorteia N_s e guarda o novo desafio no banco de dados associando ao $DevId$ que o pediu. Ao receber o desafio, o dongle confere a tag e verifica que c_S é maior do que o último aceito, valor que está salvo na NVS, garantindo que não está respondendo um desafio repetido; caso $ACK_{local} < ACK_{seq} \leq HEAD_{seq}$, em que ACK_{local} é o último valor de ACK conhecido pelo dongle, os blocos até ACK_{seq} são removidos da *flash*. Na resposta do dongle, o N_S ecoado serve para garantir que a resposta enviada é de um novo desafio. Ao avaliar a resposta, o server confere, além da tag,se o eco de c_S e de N_S corresponde ao desafio guardado no banco de dados, se ele ainda não foi usado e se não expirou; caso positivo, envia bloco ACK com flags zeradas e o token JWT.

Para o cálculo da tag, é usada a chave K_{auth} para gerar o HMAC-SHA256 segundo a equação 1, que é truncado em seguida para 16 bytes a fim de economizar banda na transmissão. Aqui é possível truncar devido ao fato de o server ter permissão de gerar MACs com K_{auth} no KMS, o que possibilita o servidor gerar o HMAC e comparar os primeiros 16 bytes; como o server não consegue gerar HMAC para K_{rec}, não é possível truncar o HMAC dos blocos sem perder o isolamento de responsabilidades no KMS.

No cálculo do HMAC em 1, além da chave e do payload, são adicionados no cálculo da tag alguns separadores de contexto para evitar que uma tag válida seja reaproveitada em outro contexto para um payload de mesmo valor. O termo "MALCONG"é um rótulo fixo para esse projeto, o *tipo* representa o tipo de bloco que está sendo calculada no MAC. A

figura 6 mostra os tipos de blocos utilizados no handshake.

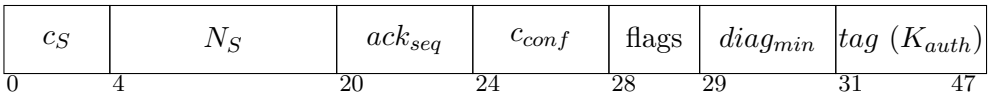
O diagrama de sequência da figura 5 mostra o fluxo de comunicação, considerando a intermdiação do app. O gatilho de handshake ocorre nas seguintes situações: boot do dongle ou nova conexão BLE, eventos que zeram a flag na RAM do dongle que indica se houve handshake, fazendo com que o dongle peça um handshake para o app a partir das mensagens que serão especificadas em 5.1; token JWT perto de expirar, o que força um novo handshake pra nova emissão de token JWT; ou âncora rejeitada, que força um novo handshake para sicronização.

Sobre a âncora de tempo, uma limitação conhecida é quando um boot fica todo seu ciclo de vida em uma conexão, o que faz com que o boot fique sem sincroniza de relógio completa.

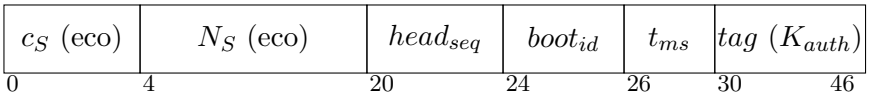
$$tag = \text{HMAC-SHA256}_{K_{auth}}(\text{"}MALCONG\text{"} \,||\, \text{tipo} \,||\, DevId \,||\, \text{payload}) \tag{1}$$

Figura 6 – Blocos utilizados no *handshake*

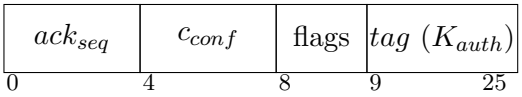
Desafio: 0x01 (47 bytes)



Resposta: 0x02 (46 bytes)



ACK: 0x04 (25 bytes)



Fonte: elaborado pelo autor.

5 Integração entre BLE e HTTPS/TLS para troca de dados entre dongle e servidor

O app se comunica com o dongle por meio de BLE. Como já foi modelado na seção 2.1.1, o BLE é um enlace sem autenticidade, confidenciabilidade e integridade caso não seja usado o padrão LESC, o que abre a possibilidade de ataques do tipo A1.2, A1.3 e A1.4 da seção 2.1.1. Por isso, o celular se comunicará apenas depois de um pareamento seguindo o padrão LESC, ajustando o firmware do dongle para aceitar apenas conexões LESC.

O método de pareamento dentre os definidos na tabela 3.1 será o Just Works, devido ao fato de o dongle não ter nenhuma capacidade de Input/Output. Isso expõe a conexão ao ataque A1.2, de *Man in The Middle*, modelado na seção 2.1.1 para um adversário do tipo AD1, que está perto do carro no momento da conexão.

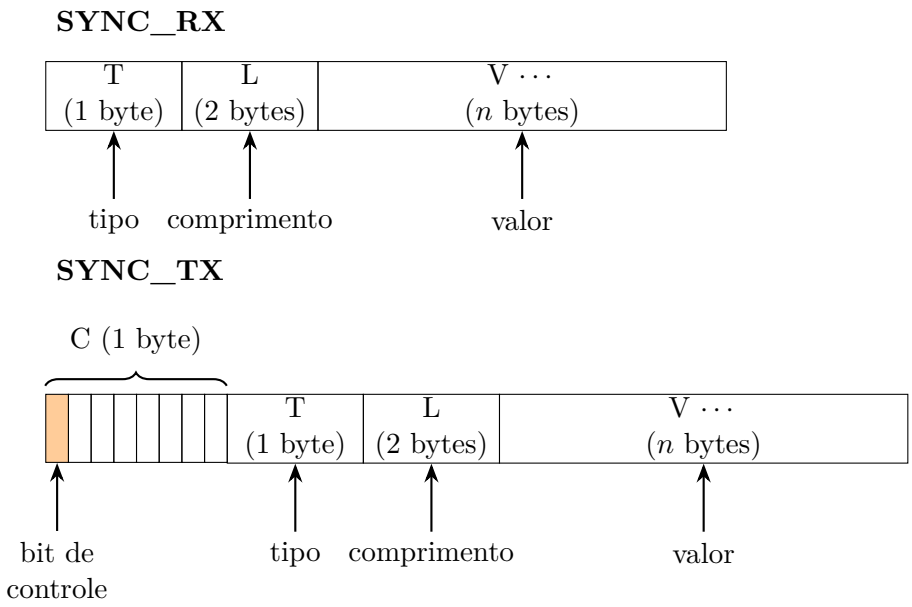
No entanto, são tomadas algumas medidas para diminuir esse risco: o dongle aceita pareamentos apenas em um intervalo de tempo definido depois de apertar um botão físico no dongle, o que deixa menos margem para o atacante conseguir interceptar a conexão; além disso, é permitido apenas um bond (pareamento) por vez, sendo que apertar o botão derruba todos os pareamentos feitos, o que desconecta um atacante. Essas medidas também evitam ataque do tipo DoS, modelados na seção 2.1.1 como A1.5

5.1 Serviços e blocos BLE para comunicação entre app e dongle

Os comandos ELM327, incluindo os comandos customizados de configuração definidos no projeto, são enviados do app para o dongle a partir do serviço NUS. Enquanto isso, os comandos de handshake e envio de blocos passam por um outro serviço criado para o projeto, chamado SYNC. Assim como o NUS, o SYNC expõem uma característica RX, na qual o dongle recebe dados do app, e uma característica TX, na qual o dongle envia dados para o app; no entanto, enquanto o NUS recebe texto, o serviço SYNC envia e recebe dados binários. Com essa separação, cada fluxo de dados entre app e dongle fica separado e com sua própria responsabilidades, ajudando na remontagem de blocos a partir dos fragmentos.

Além dos tipos de blocos definidos no handshake, existem mais alguns tipos que precisam ser usados para orquestrar a comunicação entre app e dongle, sendo todos eles apresentados na tabela 3. Quando são enviados via BLE no SYNC_RX, como convenção do projeto, os blocos são estruturados no formato TVL (type, length, value), sendo 3 bytes concatenados no início de cada bloco, o primeiro representando o tipo e o segundo o comprimento total do corpo da mensagem (sem contar o tipo e comprimento). Além disso, quando são enviados no SYNC_TX, também é necessário englobar o bloco TVL em um enquadramento com um byte de controle na frente, que vai ser o bit 0 valendo 1 quando for um início de mensagem; isso é necessário pelo fato de as mensagens do SYNC_TX poderem ser fragmentadas em vários pacotes BLE. O formato do pacote de cada característica do SYNC pode ser visto na figura 7.

Figura 7 – Formato dos dados nas características RX e TX do serviço SYNC



Fonte: elaborado pelo autor.

Tabela 3 – Tipos de blocos trocados entre servidor, app e dongle

Tipo	**Nome**	**Tamanho (bytes)**	**Parâmetros (bytes)**
0x01	Desafio	45 ou 47	c_S (4), N_S (16), ACK_{seq} (4), c_{conf} (4), $flags$ (1), $diag_min$ (2, só com o bit 0 das $flags$), tag (16)
0x02	Resposta	46	c_S (4, eco), N_S (16, eco), $HEAD_{seq}$ (4), $boot_id$ (2), t_{ms} (4), tag (16)
0x03	Bloco	392 a 1.056	seq (4), $boot_id$ (2), t_{ms} inicial (4), tamanho (2), registros (até 1.012), tag (32)
0x04	ACK	25	ACK_{seq} (4), c_{conf} (4), $flags$ (1), tag (16)
0x05	Pedido de lote	2	K (2)
0x06	Erro	1	código (1)
0x07	Aviso de novo bloco	13	$HEAD_{seq}$ (4), $flags$ (1), $DevID$ (8)

Os blocos do handshake de Desafio (0x01) e ACK (0x04), ao chegarem do servidor no app, são repassados diretamente para SYNC_RX no formato binário ou, caso necessário, são antes decodificados a partir da string base64. Enquanto isso, os blocos de Resposta (0x02) e de Envio de Blocos (0x03) são construídos no dongle e, quando chegam no app via SYNC_TX, são diretamente enviados pelo app para o servidor no seu respectivo endpoint, havendo codificação para string base64 quando necessário.

Quando o dongle dá flush em um bloco para a *flash*, ele envia um aviso de novo bloco (0x07) para o app. Quando esse aviso chega, caso o app já tenha disponível um valor de ACK_{seq} válido, ele calcula o valor de quantos blocos estão pendentes de envio a partir de $HEAD_{seq} - ACK_{seq}$, sendo o $HEAD_{seq}$ recebido via aviso. As flags desse bloco de aviso podem ser vistas na tabela 4. Caso o app ainda não tenha um handshake válido, o dongle solicita para o app que faça um pedido de desafio para o servidor, iniciando o processo de handshake.

Tabela 4 – *Flags* do aviso de novo bloco

Bit	**Significado**	**Quando liga**	**Quando desliga**	**O que o app faz**
0	Sem *handshake* nesta conexão	A cada conexão BLE nova (*flag* em RAM, zerada ao conectar)	Quando chega um ACK com c_{conf} igual ao c_S que o dongle respondeu	Faz o *handshake* antes de qualquer pedido de lote
1	Sem *handshake* neste *boot*	A cada *boot*	Igual ao bit 0	Faz o *handshake*, para o *boot* ganhar uma âncora de tempo
2	*Flash* acima de 80%	Quando a ocupação da *flash* passa de 80%	Quando cai abaixo disso, depois que ACKs liberam espaço	Prioriza o envio: pede lotes de 32 em seguida e pode avisar o usuário
3 a 7	Reservados	—	—	Ignora

Quando a conta de pendentes no app chegar em N_lote, ou quando se passar $N_lote * T_Bloco$ segundos desde o lote de blocos enviados, o app envia um bloco de Pedido de Lote (0x05) para o dongle via SYNC_RX. Como a maioria das configurações tem T_max de 100ms ou 200ms, um bloco se completa em aproximadamente 10s ($T_bloco = 10s$), então considera-se $N_lote = 6$ para enviar 6 blocos a cada 1 minuto. A quantidade de blocos pedida na mensagem é sempre o mínimo entre o número de pendentes e a quantidade de máxima de blocos aceitadas na requisição POST para o servidor, que é 32 blocos.

É importante notar que o MTU do BLE não suporta um bloco inteiro, que fica negociado em torno de 272 bytes. O bloco completo, como mostra a figura 8 terá 1059 bytes, sendo 1 bytes com o tipo, 2 bytes com o comprimento, 1024 bytes de dados e, por fim, 32 bytes de tag. Os bytes do CRC são conferidos antes de o dongle enviar os blocos via BLE, então não aparecem aqui. Para enviar os blocos então, o app primeiro consolida no formato TVL e, em seguida, divide em tamanhos iguais de $MTU - 1$ bytes, adicionando o bytes de controle, sendo que o bit 0 do byte de controle vale 1 somente para no primeiro fragmento de cada bloco da flash. Esses blocos são então mandados via BLE para o app, que remonta o bloco original da flash. Quando os blocos chegam no app, são enviados via

requisição POST. A resposta dessa requisição é um bloco ACK (0x04) que contém o novo valor de ACK_{seq}, que é usado para atualizar o app e o dongle sobre quais blocos foram reconhecidos pelo servidor.

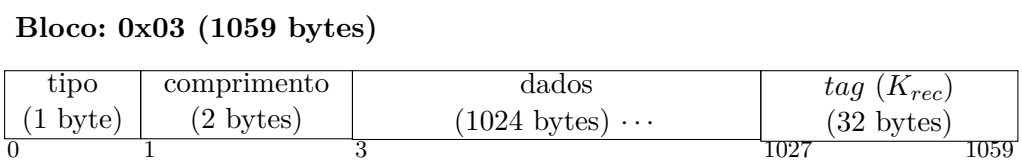
O bloco de erro carrega um byte que pode definir diferentes tipos de erros. Os erros e suas mitigações são mostrados na tabela 5.

Tabela 5 – Erros e respostas de falha entre dongle, app e servidor

Sentido	**Código**	**Quando**	**O que é feito**
Dongle → app	0x01 tag inválida	Desafio ou ACK cuja tag não confere	O app descarta a mensagem e pede um desafio novo.
Dongle → app	0x02 c_S antigo	Desafio com c_S menor ou igual ao último aceito (desafio velho ou reapresentado)	O app pede um desafio novo ao servidor
Dongle → app	0x03 ACK_{seq} inválido	ACK com ACK_{seq} menor que o do dongle ou maior que o $head_{seq}$	O app não reescreve aquele ACK; o próximo ACK ou desafio corrige a fronteira
Dongle → app	0x05 sem sessão	Pedido de lote antes do ACK de abertura	O app faz o *handshake* ou reescreve o ACK de abertura se ainda o tiver
App → dongle	0x04 mensagem incompleta	Salto no contador de fragmentos, ou 2 s sem o próximo fragmento	O dongle volta o cursor de envio para $ACK_{seq} + 1$
App → dongle	0x07 reenviar desde a fronteira	POST que falhou de vez, ou ACK com ACK_{seq} abaixo do último *seq* do lote	O dongle volta o cursor de envio para $ACK_{seq} + 1$
Servidor → app	200 com ACK e $ACK_{seq} \geq$ último *seq* do lote	Lote aceito	O app escreve o ACK no dongle e segue
Servidor → app	200 com ACK e $ACK_{seq} <$ último *seq* do lote	Lote interrompido: tag inválida, salto não declarado ou blocos fora de ordem	O app escreve o ACK no dongle, manda Erro 0x07 e descarta o lote
Servidor → app	401	Token ausente, inválido ou expirado	O app faz um *handshake* novo e reenvia o lote

Sentido	**Código**	**Quando**	**O que é feito**
Servidor → app	403	Token emitido para outro DevId	Bug do app: descarta o token e faz o *handshake*
Servidor → app	409	No POST de resposta: nonce desconhecido, vencido ou já usado, ou c_S antigo	O app pede um desafio novo
Servidor → app	429	Limite de taxa ou lote acima de 32 blocos	O app espera o Retry-After e reenvia com K menor
Servidor → app	5xx, 503 ou falha de rede	Servidor indisponível ou sobrecarregado	O app tenta o mesmo lote com espera exponencial de 1 s a 5 min; o próximo K cai pela metade

Figura 8 – Estrutura do bloco completo de dados



Fonte: elaborado pelo autor.

Para permitir mais eficiência, o dongle usa uma estratégia de cursor de envio para enviar novos blocos para o app antes de receber o ACK dos anteriores. Quando o app pede por mais blocos, mesmo o ACK ainda não tendo chegado dos blocos anteriores, o dongle manda para o app via BLE, que já lê e remonta os fragmentos para ter os blocos prontos para enviar ao servidor. Assim que o post é concluído, o próximo lote pode ser enviado sem esperar uma leitura do BLE, garantindo maior eficiência. Caso o a requisição para o servidor falhe, o app manda um bloco de erro com parâmetro 0x07 e o dongle volta o cursor de envio para o primeiro bloco ainda não reconhecido pelo servidor, que é o $ACK_{seq} + 1$, e o app descarta o pacote lido anteriormente. No fim, nada se perde, pois a flash só é limpa quando o ACK é recebido

6 Implementação do servidor

O servidor será um VPS com uma API em GoLang que responderá as requisições de desafio para o dongle feitas pelo app, processará e validará as respostas desses desafios, e também receberá os blocos de dados medidos pelo dongle vindos do app. A ideia de usar

GoLang é permitir rápido processamento das requisições e capacidade maior de carga de requisições.

A API terá 3 endpoints principais:

- POST **/v1/dongles/{DevId}/desafio**: Pedido de desafio, retorna bloco de desafio no formato binário (*application/octet-stream*)
- POST **/v1/dongles/{DevId}/desafio/resposta**: Processa a resposta ao desafio e retorna JSON contendo um bloco ACK em string base64 e um token JWT válido para envio de blocos.
- POST **/v1/dongles/blocos**: endpoint que recebe blocos em lote no formato de JSON em lista de strings base64.

O banco de dados será PostgreSQL, com TimescaleDB e PostGIS para armazenar os dados de telemetria e localização, respectivamente.

Para simplificar, no mesmo VPS, mas com configurações de acesso mais restrita, estará um servidor KMS usando HashiCorp Vault para gerenciar as chaves usadas nos HMACs de autenticação e de blocos de dados (K_{auth} e K_{rec}, respectivamente), e também a chave privada para assinar os tokens JWT que permitirão que o app envie blocos de maneira autenticada, evitando possível sobrecarga no endpoint de recebimento de blocos.

No banco, deve haver:

- Uma tabela que associa $DevId$ ao atual ACK_{seq} para saber até que bloco foi reconhecido
- Uma tabela que guarda os blocos brutos recebidos
- Uma tabela que registra os boots registrados e hora real calculada para cada um
- Uma tabela com a decodificação do bloco em posições ao longo do tempo (Hypertable do TimescaleDB)
- Uma tabela com a decodificação do bloco em PIDs, havendo uma linha para cada PID do Bloco. Cada linha é associada a com o bloco bruto por seq (Hypertable do TimescaleDB)
- Uma tabela com os desafios e quando foram respondidos, para ter um tracking dos nonces

O endpoint de blocos é autenticado com um token JWT com $sub = DevId$ e expiração de 30 minutos. Ele deve receber um lote de blocos, verificar assinatura e expiração do token e, caso esteja válido, verificar o HMAC, decodificar os base64 e registrar, na mesma transção:

- o bloco bruto
- o novo valor de ACK_{seq} para o DevId
- as linhas nas tabelas de PID e de posições ao longo do tempo, que são decodificadas a partir do bloco bruto

Referências